[Lecture Notebook] Functions

MGS3001 Python Programming for Business, Spring 2025

Chad (Chungil Chae)

Tue, 27 May 2025

code_05_00_myfunction.py

```
# code_05_00_myfunction.py
# 5.1: Introduction to Function

# This program demonstrates a function.

# First, we define a function named message.
def mysum(num1, num2):
    mysum = num1 + num2
    print(mysum)

def myaver(num1, num2):
    myaver = (num1 + num2) / 2
    print(myaver)

# Call the message function.
mysum(24, 25)
myaver(87, 88)
```

code_05_01_function_demo.py

```
# code_05_01_function_demo.py
# 5.2: Defining and Calling a Void Function

# This program demonstrates a function.
```

```
# First, we define a function named message.
def message():
    print('This is Chad,')
    print('An assistant professor of WKU.')

# Call the message function.
message()
```

7 **code_05_02_two_functions.py**

```
# code_05_02_two_functions.py
# 5.2: Defining and Calling a Void Function

# This program has two functions. First we define the main function.

# Define the main function.
def main():
    print('I have a message for you.')
    message()
    print('Goodbye!')

# next we define the message function.
def message():
    print('I am Chad,')
    print('A professor of MGS3001')

# Call the main function
main()
```

8 **code_05_03_acme_dryer.py**

```
# code_05_03_acme_dryer.py
#5.3: Designing a Program to Use Functions

# This program displays step-by-step instructions for disassembling an Acme dryer

# The main function performs the program's main logic.
def main():
    # Display the start-up message.
```



```
    startup_message()
    input('Press Enter to see Step 1.')
    step1()
    # Display step 2.
    input('Press Enter to see Step 2.')
    step2()
    # Display step 3.
    input('Press Enter to see Step 3.')
    step3()
    # Display step 4.
    input('Press Enter to see Step 4.')
    step4()

# The startup_message function displays the program's initial message on the screen.
def startup_message():
    print('This program tells you how to')
    print('disassemble an ACME laundry dryer.')
    print('There are four steps in the process.')
    print('')

# For step1
def step1():
    print('Step 1: Unplug the dryer and')
    print('move it away from the wall.')
    print('')
# For step2
def step2():
    print('Step 2: Remove the six screws')
    print('from the back of the dryer.')
    print('')
# For step3
def step3():
    print('Step 3: Remove the back panel')
    print('from the dryer')
    print('')
# For step4
def step4():
    print('Step 4: Pull the top of the')
    print('dryer straight up.')

# Call the main function to begin the program.
main()
```


9 ## code_05_04_01_bad_local.py

```
# code_05_04_01_bad_local.py
#5.5: Passing Arguments to Functions

#

# Definition of the main function.
def main():
    get_name()
    print('Hello', name) # This causes an error!

# Definition of the get_name function.
def get_name():
    name = input('Enter your name: ')

# Call the main function.
main()
```

10 ## code_05_04_02_bad_local_fix.py

```
# code_05_04_02_bad_local_fix.py
#5.4: Local Variables

#

# Definition of the main function. and pass value to print_name
def main():
    name = input('Enter your name: ')
    print_name(name)

# Define print_name
def print_name(name_from_main):
    print('Hello', name_from_main)

# Call the main function.
main()
```


11 ## code_05_05_birds.py

```
# code_05_05_birds.py
#5.4: Local Variables

# This program demonstrates two functions that have local variables with the same name.
def main():
    # Call the texas function.
    texas()
    # Call the california function.
    california()

# Definition of the texas function. It creates a local variable named birds.
def texas():
    birds = 5000
    print('texas has', birds, 'birds.')

# Definition of the california function. It also creates a local variable named birds.
def california():
    birds = 8000
    print('california has', birds, 'birds.')

# Call the main function.
main()
```

12 ## code_05_06_pass_arg.py

```
# code_05_06_pass_arg.py
#5.5: Passing Arguments to Functions

# This program demonstrates an argument being passed to a function.

# Define main function
def main():
    value = 5
    show_double(value)

# The show_double function accepts an argument and displays double its value.

# Define show_double
def show_double(number):
```



```
    result = number * 2
    print(result)

# Call the main function
main()
```

code_05_07_cups_to_ounces.py

```
# code_05_07_cups_to_ounces.py
#5.5: Passing Arguments to Functions

# This program converts cups to fluid ounces.

## Define main function
def main():
    # display the intro screen.
    intro()
    # get the number of cups
    cups_needed = int(input('Enter the number of cups: '))
    # Convert the cups to ounces.
    cups_to_ounces(cups_needed)

## The intro function displays an introductory screen.
def intro():
    print('This program converts measurements in cups to fluid ounces.')
    print('For your reference the formula is: ')
    print('1 cup = 8 fluid ounces')
    print()

## The cups_to_ounces function accepts a number of cups and displays the equivalent number of oun
def cups_to_ounces(cups):
    ounces = cups * 8
    print('That converts to', ounces, 'ounces.')

## Call the main function
main()
```


14 ## code_05_08_01_multiple_args.py

```
# code_05_08_01_multiple_args.py
#5.5: Passing Arguments to Functions

#

def main():
    print('The sum of 12 and 45 is ')
    show_sum(12, 45)

# show_sum
def show_sum(num1, num2):
    result = num1 + num2
    print(result)

# main()
main()
```

15 ## code_05_08_02_multiple_args.py

```
# code_05_08_02_multiple_args.py
#5.5: Passing Arguments to Functions

#

def main():
    print('Input 2 numbers')
    n1 = int(input('Enter 1st number: '))
    n2 = int(input('Enter 2nd number: '))
    show_sum(n1, n2)

# show_sum
def show_sum(num1, num2):
    result = num1 + num2
    print('the sum is', result)

# main()
main()
```

16 ## code_05_09_string_args.py

```
# code_05_09_string_args.py
#5.5: Passing Arguments to Functions

#     2

def main():
    first_name = input('Enter your first name: ')
    last_name = input('Enter your last name: ')
    print('Your name reversed is')
    reverse_name(first_name, last_name)

def reverse_name(first, last):
    print(last, first)

main()
```

17 ## code_05_10_change_me.py

```
# code_05_10_change_me.py
#5.5: Passing Arguments to Functions

#

def main():
    value = 99
    print('The value is', value)
    change_me(value)
    print('Back in main value is', value)

def change_me(arg):
    print('I am change the value.')
    arg = 0
    print('Now the valu is', arg)

main()
```


18 ## code_05_11_keyword_args.py

```
# code_05_11_keyword_args.py
#5.5: Passing Arguments to Functions

#

def main():
    # Show the amount of simple interest, using 0.01 as
    # interest rate per period, 10 as the number of periods,
    # and $10,000 as the principal.
    show_interest(rate = 0.01, periods = 10, principal = 10000.0)

# The show_interest function displays the amount of simple interest for a given principal,
# interest rate, per period, and number of periods

def show_interest(principal, rate, periods):
    interest = principal * rate * periods
    print('The simple interest will be $',
    format(interest, ',.2f'),
    sep = "")

# Call the main function
main()
```

19 ## code_05_12_keyword_string_args.py

```
# code_05_12_keyword_string_args.py
#5.5: Passing Arguments to Functions

# This program demonstrates passing two strings as
# keyword arguments to a function.
def main():
    first_name = input('Enter your first name: ')
    last_name = input('Enter your last name: ')
    print('Your name reversed is')
    reverse_name(last=last_name, first=first_name)

def reverse_name(first, last):
    print(last, first)
```



```
# Call the main function.
main()
```

20 **code_05_13_global1.py**

```
# code_05_13_global1.py
#5.6: Global Variables and Global Constants

# Create a global variable
my_value = 10

# The show_value function prints the value of the global variable
def show_value():
    print(my_value)

# Call the show_value function
show_value()
```

21 **code_05_14_global2.py**

```
# code_05_14_global2.py
#5.6: Global Variables and Global Constants

# Create a global variable
number = 0

def main():
    global number
    number = int(input('Enter a number: '))
    show_number()

def show_number():
    print('The number you entered is', number)

# Call the main function
main()
```


code_05_15_retirement.py

```
# code_05_15_retirement.py
#5.6: Global Variables and Global Constants

# The following is used as a global constant the contribution rate.
CONTRIBUTION_RATE = 0.05

def main():
    gross_pay = float(input('Enter the gross pay: '))
    bonus = float(input('Enter the amount of bonuses: '))
    show_pay_contrib(gross_pay)
    show_bonus_contrib(bonus)

# The show_pay_contrib function accepts the gross pay as an argument and displays the retirement
# contribution for that amount of pay.
def show_pay_contrib(gross):
    contrib = gross * CONTRIBUTION_RATE
    print('Contribution for gross pay: $',
    format(contrib, ',.2f'), sep='')

# The show_bonus_contrib function accepts the bonus amount as an argument and displays the
# retirement contribution for that amount of pay.
def show_bonus_contrib(bonus):
    contrib = bonus * CONTRIBUTION_RATE
    print('Contribution for bonuses: $',
    format(contrib, ',.2f'), sep='')

# Call the main function.
main()
```

code_05_16_random_numbers1.py

```
# code_05_16_random_numbers1.py
#5.7: Introduction to Value-Returning Functions: Generating Random Numbers

# This program displays a random number in the range of 1 through 10
import random

def main():
    number = random.randint(1, 10) # Get a random number
```



```
    print('The number is', number) # Display the number

# Call the main function.
main()
```

code_05_17_random_numbers2.py

```
# code_05_17_random_numbers2.py
#5.7: Introduction to Value-Returning Functions: Generating Random Numbers

# This program displays five random numbers in the range of 1 through 100.
import random

def main():
    for count in range(5):
        number = random.randint(1, 100)
        print(number)

main()
```

code_05_18_random_numbers3.py

```
# code_05_18_random_numbers3.py
#5.7: Introduction to Value-Returning Functions: Generating Random Numbers

import random

def main():
    for count in range(5):
        print(random.randint(1,100))

main()
```

code_05_19_dice.py


```
# code_05_19_dice.py
#5.7: Introduction to Value-Returning Functions: Generating Random Numbers

# this program the rolling of dice
import random

# Constants for the minimum and maximum random numbers
min = 1
max = 6

def main():
    again = 'y' # create a variable to control the loop

    # simulate rolling the dice
    while again == 'y' or again == 'Y':
        print('Rolling the dice ...')
        print('Their values are: ')
        print(random.randint(min, max))
        print(random.randint(min, max))

        # Do another roll of the dice?
        again = input('Roll them again? (y for Yes): ')

main()
```

27 **code_05_20_coin_toss.py**

```
# code_05_20_coin_toss.py
#5.7: Introduction to Value-Returning Functions: Generating Random Numbers

import random

# Constants
head = 1
tails = 2
tosses = 10

def main():
    for toss in range(tosses):
        if random.randint(head, tails) == head:
            print('Heads')
        else:
```



```
            print('Tails')

main()
```

code_05_21_total_ages.py

```
# code_05_21_total_ages.py
#5.8: Writing Your Own Value-Returning Functions

# This program uses the return value of a function

def main():
    first_age = int(input('Enter your age: ')) # Get the user's age
    second_age = int(input("Enter your best friend's age:" )) # Get the user's best friend's age
    total = sum(first_age, second_age) # Get the sum(first_age, second_age)
    print('Together your are', total, 'years old.')

# The sum function accepts two numeric arguments and returns the sum of those arguments
def sum(num1, num2):
    result = num1 + num2
    return result

# Call the main function.
main()
```

code_05_22_sale_price_0_empty.py

```
# code_05_22_sale_price_0_empty.py
#5.8: Writing Your Own Value-Returning Functions

# This program calculates a retail item's sale price.


# DISCOUNT_PERCENTAGE is used as a global constant for the discount percentage.

# The main function.
    # Get the item's regular price. Get regular price from user
    # Calculate the sale price.
    # Display the sale price.
```



```
# The get_regular_price function

# discount function

# Call the main function.
```

30 **code_05_22_sale_price_1.py**

```
# code_05_22_sale_price_1.py
#5.8: Writing Your Own Value-Returning Functions

# This program calculates a retail item's sale price.


# DISCOUNT_PERCENTAGE is used as a global constant for the discount percentage.
DISCOUNT_PERCENTAGE = 0.20

# The main function.
def main():
    # Get the item's regular price. Get regular price from user
    regular_price = 0
    # Calculate the sale price.
    sale_price = 0
    # Display the sale price.

# The get_regular_price function

# discount function

# Call the main function.
main()
```

31 **code_05_22_sale_price_2.py**

```
# code_05_22_sale_price_2.py
#5.8: Writing Your Own Value-Returning Functions

# This program calculates a retail item's sale price.
```



```
# DISCOUNT_PERCENTAGE is used as a global constant for the discount percentage.
DISCOUNT_PERCENTAGE = 0.20

# The main function.
def main():
    # Get the item's regular price. Get regular price from user
    regular_price = get_regular_price()
    print(regular_price)
    # Calculate the sale price.
    sale_price = 0
    # Display the sale price.

# The get_regular_price function
def get_regular_price():
    regular_price = float(input("Enter the item's regular price: "))
    return regular_price
# discount function

# Call the main function.
main()
```

32 **code_05_22_sale_price_3.py**

```
# code_05_22_sale_price_3.py
#5.8: Writing Your Own Value-Returning Functions

# This program calculates a retail item's sale price.


# DISCOUNT_PERCENTAGE is used as a global constant for the discount percentage.
DISCOUNT_PERCENTAGE = 0.20

# The main function.
def main():
    # Get the item's regular price. Get regular price from user
    regular_price = get_regular_price()
    #print(regular_price)
    # Calculate the sale price.
    sale_price = regular_price - discount(regular_price)
    print(sale_price)
    # Display the sale price.
```



```
# The get_regular_price function
def get_regular_price():
    regular_price = float(input("Enter the item's regular price: "))
    return regular_price

# discount function
def discount(price):
    return price * DISCOUNT_PERCENTAGE

# Call the main function.
main()
```

33 **code_05_22_sale_price_4_final.py**

```
# code_05_22_sale_price_4_final.py
#5.8: Writing Your Own Value-Returning Functions

# This program calculates a retail item's sale price.

# DISCOUNT_PERCENTAGE is used as a global constant for the discount percentage.
DISCOUNT_PERCENTAGE = 0.20

# The main function.
def main():
    # Get the item's regular price. Get regular price from user
    regular_price = get_regular_price()
    #print(regular_price)
    # Calculate the sale price.
    sale_price = regular_price - discount(regular_price)
    #print(sale_price)
    # Display the sale price.
    print('The sale price is $', format(sale_price, ',.2f'))

# The get_regular_price function
def get_regular_price():
    regular_price = float(input("Enter the item's regular price: "))
    return regular_price

# discount function
def discount(price):
    return price * DISCOUNT_PERCENTAGE
```



```
# Call the main function.
main()
```

34 code_05_22_sale_price_final.py

```
# code_05_22_sale_price_final.py
#5.8: Writing Your Own Value-Returning Functions

# This program calculates a retail item's sale price.

# DISCOUNT_PERCENTAGE is used as a global constant for the discount percentage.
DISCOUNT_PERCENTAGE = 0.20

# The main function.
def main():
    # Get the item's regular price. Get regular price from user
    reg_price = get_regular_price()
    # Calculate the sale price.
    sale_price = reg_price - discount(reg_price)
    # Display the sale price.
    print('The sale price is $', format(sale_price, ',.2f'), sep='')

# The get_regular_price function prompts the user to enter an item's regular price and it returns
def get_regular_price():
    price = float(input("Enter the item's regular price: "))
    return price
    # The discount function accepts an item's price as an argument and returns the amount of the

def discount(price):
    return price * DISCOUNT_PERCENTAGE

# Call the main function.
main()
```

35 code_05_23_commision_rate.py

```
# code_05_23_commision_rate.py
#5.8: Writing Your Own Value-Returning Functions
```



```
# This program calculates a salesperson's pay at Make Your Own Music.
def main():
    # Get the amount of sales.
    sales = get_sales()
    # Get the amount of advanced pay.
    advanced_pay = get_advanced_pay()
    # Determine the commission rate.
    comm_rate = determine_comm_rate(sales)
    # Calculate the pay.
    pay = sales * comm_rate - advanced_pay
    # Display the amount of pay.
    print('The pay is $', format(pay, ',.2f'), sep='')
    # Determine whether the pay is negative.
    if pay < 0:
        print('The Salesperson must reimburse the company')

# The get_sales function gets a salesperson's monthly sales from the user and returns that value.
def get_sales():
    # Get the amount of monthly sales.
    monthly_sales = float(input('Enter the monthly sales: '))
    # Return the amount entered.
    return monthly_sales

# The get_advanced_pay function gets the amount of # advanced pay given to the salesperson and re
def get_advanced_pay():
    # Get the amount of advanced pay.
    print('Enter the amount of advanced pay, or enter 0 if no advanced pay was given.')
    advanced = float(input('Advanced pay: '))
    # Return the amount entered.
    return advanced

# The determine_comm_rate function accepts the
# amount of sales as an argument and returns the # applicable commission rate.
def determine_comm_rate(sales):
    # Determine the commission rate.
    if sales < 10000.00:
        rate = 0.10
    elif sales >= 10000 and sales <= 14999.99:
        rate = 0.12
    elif sales >= 15000 and sales <= 17999.99:
        rate = 0.14
    elif sales >= 18000 and sales <= 21999.99:
        rate = 0.16
    else:
```



```
        rate = 0.18
    # Return the commission rate.
    return rate

main()
```

code_05_24_square_root.py

```
# code_05_24_square_root.py
#5.9: The math Module

import math

def main():
    # Get a number
    number = float(input('Enter a number: '))

    # Get the square root of the number
    square_root = math.sqrt(number)

    # Display the square root
    print('The square root of', number, '0 is', square_root)

main()
```

code_05_25_hypothnuse.py

```
# code_05_25_hypothnuse.py
#5.9: The math Module

# This program calculates the length of a right # triangle's hypotenuse.
import math

def main():
    # Get the length of the triangle's two sides.
    a = float(input('Enter the length of side A: '))
    b = float(input('Enter the length of side B: '))
    # Calculate the length of the hypotenuse.
    c = math.hypot(a, b)
```



```
    # Display the length of the hypotenuse (     -    ).
    print('The length of the hypotenuse is', c)

# Call the main function.
main()
```

38 **code_05_26_circle_01.py**

```
# code_05_26_circle_01.py
#5.10: Storing Functions in Modules

# The circle module has functions that perform # calculations related to circles.
import math

def main():
    radius = float(input("Enter the radius: "))
    print('The area of the circle is', format(area(radius), ',.2f')) #
    print('The circumference of the circle is', format(circumference(radius), ',.2f')) #

# The area function accepts a circle's radius as an # argument and returns the area of the circle
def area(radius):
    return math.pi * radius**2
    # The circumference function accepts a circle's
    # # radius and returns the circle's circumference.

def circumference(radius):
    return 2 * math.pi * radius

main()
```

39 **code_05_27_rectangle_01.py**

```
# code_05_27_rectangle_01.py
#5.10: Storing Functions in Modules

# The rectangle module has functions that perform calculations related to rectangles.

def main():
    width = float(input("Enter the width: "))
```



```
    length = float(input("Enter the length: "))
    print('The area of the rectangle is', format(area(width, length), ',.2f')) #
    print('The perimeter of the rectangle is', format(perimeter(width, length), ',.2f')) #

# The area function accepts a rectangle's width and length as arguments and returns the rectangle
def area(width, length):
    return width * length

# The perimeter function accepts a rectangle's width and length as arguments and returns the rect
def perimeter(width, length):
    return 2 * (width + length)

main()
```

40 ## code_05_28_geometry

41 ### circle.py

```
# circle.py
#5.10: Storing Functions in Modules

# The circle module has functions that perform # calculations related to circles.
import math

# The area function accepts a circle's radius as an # argument and returns the area of the circle
def area(radius):
    return math.pi * radius**2
    # The circumference function accepts a circle's
    # # radius and returns the circle's circumference.

def circumference(radius):
    return 2 * math.pi * radius
```

42 ### rectangle.py

```
#5.10: Storing Functions in Modules

# The area function accepts a rectangle's width and length as arguments and returns the rectangle
def area(width, length):
    return width * length
```



```
# The perimeter function accepts a rectangle's width and length as arguments and returns the rect
def perimeter(width, length):
    return 2 * (width + length)
```

43 **code_05_28_geometry.py**

```
# code_05_28_geometry.py
#5.10: Storing Functions in Modules

import circle
import rectangle

# Constants for the menu choices
AREA_CIRCLE_CHOICE = 1
CIRCUMFERENCE_CHOICE = 2
AREA_RECTANGLE_CHOICE = 3
PERIMETER_RECTANGLE_CHOICE = 4
QUIT_CHOICE = 5

# The main function.
def main():
    # The choice variable controls the loop and holds the user's menu choice.
    choice = 0

    while choice != QUIT_CHOICE:
        # display the menu.
        display_menu()

        # Get the user's choice.
        choice = int(input('Enter your choice: '))

        # Perform the selected action.
        if choice == AREA_CIRCLE_CHOICE:
            radius = float(input("Enter the circle's radius: "))
            print('The area is', circle.area(radius))
        elif choice == CIRCUMFERENCE_CHOICE:
            radius = float(input("Enter the circle's radius: "))
            print('The circumference is',
            circle.circumference(radius))
        elif choice == AREA_RECTANGLE_CHOICE:
            width = float(input("Enter the rectangle's width: "))
            length = float(input("Enter the rectangle's length: "))
```



```
            print('The area is', rectangle.area(width, length))
        elif choice == PERIMETER_RECTANGLE_CHOICE:
            width = float(input("Enter the rectangle's width: "))
            length = float(input("Enter the rectangle's length: "))
            print('The perimeter is', rectangle.perimeter(width, length))
        elif choice == QUIT_CHOICE:
            print('Exiting the program...')
        else:
            print('Error: invalid selection.')

# The display_menu function displays a menu.
def display_menu():
    print()
    print('[MENU]')
    print('1) Area of a circle')
    print('2) Circumference of a circle')
    print('3) Area of a rectangle')
    print('4) Perimeter of a rectangle')
    print('5) Quit')

# Call the main function.
main()
```

44 **code_05_29_draw_squares.py**

```
# code_05_29_draw_squares.py
#5.11: Turtle Graphics: Modularizing Code with Functions

# The square function draws a square. The x and y parameters
# are the coordinates of the lower-left corner. The width
# parameter is the width of each side. The color parameter
# is the fill color, as a string.

import turtle

def main():
    turtle.hideturtle()
    square(100, 0, 50, 'red')
    square(-150, -100, 200, 'blue')
    square(-200, 150, 75, 'green')
    turtle.done()
```



```
def square(x, y, width, color):
    turtle.penup()
    turtle.goto(x, y)
    turtle.fillcolor(color)
    turtle.pendown()
    turtle.begin_fill()
    for count in range(4):
        turtle.forward(width)
        turtle.left(90)
    turtle.end_fill()

# Raise the pen
# Move to the specified location # Set the fill color
# Lower the pen
# Start filling

main()
```

45 **code_05_30_draw_circles.py**

```
# code_05_30_draw_circles.py
#5.11: Turtle Graphics: Modularizing Code with Functions

# The circle function draws a circle. The x and y parameters are the coordinates of the center po

import turtle

def main():
    turtle.hideturtle()
    circle(0, 0, 100, 'red')
    circle(-150, -75, 50, 'blue')
    circle(-200, 150, 75, 'green')
    turtle.done()

def circle(x, y, radius, color):
    turtle.penup() # Raise the pen
    turtle.goto(x, y - radius) # Position the turtle
    turtle.fillcolor(color) # Set the fill color
    turtle.pendown() # Lower the pen
    turtle.begin_fill() # Start filling
    turtle.circle(radius) # Draw a circle
    turtle.end_fill() # End filling
```



```
# Call the main function.
main()
```

46 **code_05_31_draw_lines.py**

```
# code_05_31_draw_lines.py
#5.11: Turtle Graphics: Modularizing Code with Functions

# The line function draws a line from (startX, startY) to (endX, endY). The color parameter is th

import turtle

# Named constrants for the triangle's position
TOP_X = 0
TOP_Y = 100
BASE_LEFT_X = -100
BASE_LEFT_Y = -100
BASE_RIGHT_X = 100
BASE_RIGHT_Y = -100

def main():
    turtle.hideturtle()
    line(TOP_X, TOP_Y, BASE_LEFT_X, BASE_LEFT_Y, 'red')
    line(TOP_X, TOP_Y, BASE_RIGHT_X, BASE_RIGHT_Y, 'blue')
    line(BASE_LEFT_X, BASE_LEFT_Y, BASE_RIGHT_X, BASE_RIGHT_Y, 'green')
    turtle.done()

def line(startX, startY, endX, endY, color):
    turtle.penup()
    turtle.goto(startX, startY)
    turtle.pendown()
    turtle.pencolor(color)
    turtle.goto(endX, endY)

main()
```


code_05_33_graphics_mod_demo

my_graphics.py

```
#5.11: Turtle Graphics: Modularizing Code with Functions

import turtle

# The square function draws a square. The x and y parameters are the coordinates of the lower-lef
def square(x,y, width, color):
    turtle.penup()
    turtle.goto(x,y)
    turtle.fillcolor(color)
    turtle.pendown()
    turtle.begin_fill()
    for count in range(4):
        turtle.forward(width)
        turtle.left(90)
    turtle.end_fill()

# The circle function draws a circle. The x and y parameters are the corrdinates of teh center po
def circle(x, y, radius, color):
    turtle.penup()
    turtle.goto(x, y - radius)
    turtle.fillcolor(color)
    turtle.pendown()
    turtle.begin_fill()
    turtle.circle(radius)
    turtle.end_fill()

# The line function draws a line from (startX, startY) to (endX, endY). The color parameter is th
def line(startX, startY, endX, endY, color):
    turtle.penup()
    turtle.goto(startX, startY)
    turtle.pendown()
    turtle.pencolor(color)
    turtle.goto(endX, endY)
```

code_05_33_graphics_mod_demo.py

```
# code_05_33_graphics_mod_demo.py
#5.11: Turtle Graphics: Modularizing Code with Functions
```



```
import turtle
import my_graphics

# Named constants
X1 = 0
Y1 = 100
X2 = -100
Y2 = -100
X3 = 100
Y3 = -100
RADIUS = 50

def main():
    turtle.hideturtle()

    # Draw a square
    my_graphics.square(X2,Y2,(X3-X2),'gray')

    # Draw a circles
    my_graphics.circle(X1, Y1, RADIUS,'blue')
    my_graphics.circle(X2, Y2, RADIUS,'red')
    my_graphics.circle(X3, Y3, RADIUS,'green')

    # Draw a lines
    my_graphics.line(X1,Y1,X2,Y2,'black')
    my_graphics.line(X1,Y1,X3,Y3,'black')
    my_graphics.line(X2,Y2,X3,Y3,'black')

    turtle.done()

main()
```

